PAGE 1 — ROW vs COLUMN STORAGE + RELATIONAL MODEL

1. ROW-ORIENTED STORAGE

- Data is stored row-wise, i.e., all column values of a row are stored together on disk.
- Efficient for reading or updating complete records.
- Good for point lookups using primary key.
- Commonly used in OLTP systems (transactional databases).
- Suitable when frequent insert, update and delete operations happen on individual rows.
- For analytical queries (e.g., SUM(amount) over 100M records), it may scan many unnecessary column values which can be slower.

Example Table : Customers

customer_id	name	city	amount
101	Rahul	Delhi	500
102	Priya	Pune	800

Row-wise storage on disk

101	Rahul	Delhi	500
102	Priya	Pune	800
...			

All column values of a row stored together

2. COLUMN-ORIENTED STORAGE

- Data is stored column-wise, i.e., values of each column are stored together on disk.
- Suitable for analytical queries (SUM, AVG, GROUP BY) where only a few columns are read (column pruning).
- Supports better compression as similar data types are stored together.
- Enables vectorized execution and faster scans for large datasets.
- Widely used in data warehouses and OLAP systems (e.g., Redshift, BigQuery, Snowflake, ClickHouse, Parquet).
- Reconstructing many full records and frequent single-row updates may be more expensive compared to row storage.

Column-wise storage on disk

customer_id	name	city	amount
101	Rahul	Delhi	500
102	Priya	Pune	800
...	...	...	...

Values of each column stored together

3. ROW vs COLUMN STORAGE - COMPARISON

Aspect	Row-Oriented Storage	Column-Oriented Storage
Data Layout	All column values of a row stored together.	Values of each column stored together.
Best Workload	OLTP (frequent inserts/updates, point lookups, complete record access)	OLAP / Analytical (aggregate queries, column pruning, large scans)
Data Read	Reads entire row even if only a few columns are needed.	Reads only required columns (column pruning).
Compression	Lower compression (different data types stored together).	Higher compression (similar data types stored together).
Updates	Faster for single row updates.	Slower for frequent single row updates (may need to rewrite column data).
Examples	MySQL, PostgreSQL, SQL Server (default row store)	Redshift, BigQuery, Snowflake, ClickHouse, Parquet, ORC (columnar formats)

Important : Columnar analytical storage (Parquet / Redshift / warehouse engines) ≠ wide-column NoSQL model (Cassandra / HBase). Storage design and data model differ. SQL databases may support both row and column indexes.

4. RELATIONAL MODEL - KEY CONCEPTS

- A table in relational model is called a relation.
- Each row is a tuple.
- Each column is an attribute.
- Relation **schema** defines the structure of the relation (column names, data types, constraints).
- Relation instance is the actual data (rows) in the table at a given point in time.
- Degree of a relation = number of columns (attributes).
- Cardinality of a relation = number of rows (tuples).
- Domain = set of allowed values for an attribute.
- NULL = unknown or missing value.

Customers (Relation)

customer_id (PK)	name	city
101	Rahul	Delhi
102	Priya	Pune

1 — Many

Relationship

Orders (Relation)

order_id (PK)	customer_id (FK)	amount
01	101	500
02	102	800

Relation Schema (Structure)
Customers(customer_id, name, city)

Degree (Customers) = 3 columns

Cardinality (Customers) = 2 rows

Relation Schema (Structure)
Orders(order_id, customer_id, amount)

Degree (Orders) = 3 columns

Cardinality (Orders) = 2 rows

5. RELATIONAL INTEGRITY & OPERATIONS

1 Entity Integrity
- Primary key values must be unique and non-null.
- Each row is uniquely identified by a primary key.

2 Referential Integrity
- Foreign key value must match an existing primary key value in the parent table (or be NULL if allowed).
- Ensures valid relationships between tables.

3 Domain Integrity
- Attribute values must come from a valid domain (e.g., data type, range, format).
- Implemented using data types, CHECK constraints, etc..

4 Relational Operations
- SELECT – filter rows based on condition.
- PROJECT – select specific columns.
- JOIN – combine rows from two or more relations.
- UNION – combine result sets (with same structure).

Important Points
- Relational model is a logical model and independent of physical storage (row or column oriented).
- Tables are treated as unordered sets of rows (no implicit row ordering).
- Use primary and foreign keys to maintain data integrity.
- NULL represents unknown or missing value (not zero or empty string).

Quick Recall

ROW = retrieve / edit a record
COLUMN = aggregate few fields
RELATION = table
TUPLE = row
DEGREE = number of columns
CARDINALITY = number of rows

PAGE 2 INDEXING, CLUSTERED INDEXES & SQL VIEWS

1. INDEXING — WHAT, WHY & COST

An **index** is an auxiliary structure mapping search keys to rows. It can avoid a full scan for **selective filters, joins, ranges and sorting**.

How a B+ tree works

Root: 50 ↘ Leaves: 10, 25, 50, 80

Ordered leaves allow both equality lookups and range scans.

Read speed is not free: indexes occupy storage and must be maintained for INSERT / UPDATE / DELETE.

Index types to recognize

Type	Where useful
B-tree / B+ tree	Equality, range, ordering
Hash	Equality (engine-dependent)
Bitmap	Analytical low-cardinality filters
Composite	Multi-column predicates
Full-text	Text search, not ordinary LIKE alone

2. CLUSTERED vs NON-CLUSTERED

Aspect	Clustered (SQL Server style)	Non-clustered
Layout	Leaf level holds actual data rows, organized by clustering key	Separate index with keys + row locator / clustered key
Count	At most **one** per table	Typically **many** per table
Good for	Ranges, clustered-key searches	Selective searches, joins, covering queries
Cost	Influences row organization	More storage and write maintenance

Interview traps: PRIMARY KEY ≠ always clustered. Clustered keys need not be unique in SQL Server (an internal uniquifier may be added). Exact behavior differs by DBMS. Spark/Databricks **clustering** is not this same index structure.

3. CHOOSE AN INDEX FROM THE QUERY

Composite / covering index

```
CREATE INDEX ix_orders_customer_date
ON orders (customer_id, order_date);
```

For WHERE customer_id = 101 AND order_date >= '2026-10-01', this can support a seek/range scan.

- **Leftmost prefix:** index (customer_id, order_date) usually benefits customer_id alone; order_date alone may not use it efficiently.
- **Covering:** index includes all columns needed by query, potentially avoiding extra table lookups.
- **Low selectivity:** a full table scan can be faster than an index (e.g. half the rows match).

4. QUERY PLAN & PRACTICAL TUNING

Before adding an index

- Run **EXPLAIN / EXPLAIN ANALYZE** (where supported).
- Inspect scan vs seek, estimated/actual row counts, join type, I/O and sorts.
- Check selectivity, frequently queried filters and sort/group columns.
- Measure SELECT gains vs slower writes & index maintenance.

SARGable filters

```
-- Often less index-friendly:
WHERE DATE(order_ts) = '2026-10-01'

-- Usually better:
WHERE order_ts >= '2026-10-01'
  AND order_ts <  '2026-10-02'
```

Avoid wrapping an indexed column in functions when a range predicate expresses the same logic.

5. SQL VIEW — VIRTUAL / LOGICAL LAYER

A regular **VIEW** stores a named SQL definition, **not a separate copy of query results**. It simplifies joins/business logic and can restrict exposed columns/rows with correct permissions.

```
CREATE VIEW v_completed_orders AS
SELECT order_id, customer_id, amount
FROM orders WHERE status = 'COMPLETE';

SELECT * FROM v_completed_orders;
```

- **Upside:** reuse logic, abstraction, consistent calculations, controlled access.
- **Limit:** a normal view does not automatically cache or speed a complex query.
- **Updatable?** Only for eligible views; complex joins/aggregations often block direct updates.

6. MATERIALIZED VIEW & DESIGN CHOICES

Regular view

Stored SQL → reads underlying data at query time.
Current data, underlying query cost.

Materialized view

Stored/precomputed result → query reads stored output.
Faster repeated analysis; refresh/staleness cost.

```
-- Illustrative syntax; supported forms vary:
CREATE MATERIALIZED VIEW mv_daily_sales AS
SELECT order_date, SUM(amount) AS revenue
FROM orders GROUP BY order_date;
```

Production decision: If a 30-second dashboard query runs every minute, consider a materialized view or aggregate table. Define refresh cadence, backfill, freshness SLA, access control and reconciliation with source.

INTERVIEW MENTAL MODEL: 1) Query pattern → 2) Data volume / selectivity → 3) Index or layout → 4) EXPLAIN & benchmark → 5) Write/refresh cost. **Never say an index or ordinary view always speeds a query.**

PAGE 3 TOP 6 REAL-WORLD DATA ENGINEERING SCENARIOS

Q1. 100M ORDERS — QUERY IS SLOW

Question: Orders table has 100M rows. Dashboard filters by customer_id and a one-month order_date range. How would you tune it?

Expected interview answer

- Capture SQL, latency, row counts, **EXPLAIN plan**; check whether query scans the whole table.
- On a row-store, evaluate a composite B-tree **(customer_id, order_date)**, with customer_id first for this predicate.
- Keep filters sargable, use needed columns, and consider a covering index for frequent requests.
- For a columnar warehouse, test **partition pruning, clustering and statistics** instead of assuming SQL Server indexes exist.
- Benchmark before/after and check extra index storage + INSERT/UPDATE overhead.

Follow-up: Why not index every column? → Write amplification, storage, optimizer complexity, diminishing returns.

Q2. ROW STORE OR COLUMN STORE?

Question: One application updates single customer records; another aggregates sales over 2 billion events. Same storage design?

Expected interview answer

- **OLTP:** row-oriented layout is generally efficient for point lookups and frequent row mutations.
- **Analytics:** columnar storage scans only relevant columns, compresses similar values and accelerates aggregates.
- Separate operational and analytical workloads using CDC / batch into a warehouse or lakehouse if needed.
- Design for freshness SLA, data quality, reconciliation and independent compute scaling.
- Test realistic workloads: a column store isn't automatically best for every query.

Follow-up: Is Cassandra a columnar OLAP warehouse? → No: wide-column NoSQL ≠ columnar analytical storage.

Q3. INDEXES MADE INGESTION SLOWER

Question: After adding six secondary indexes to a high-write Orders table, inserts slow down sharply. What do you do?

Expected interview answer

- Confirm regression using write metrics, latency, plans, lock waits and index sizes.
- Each insert must also maintain affected index entries; too many indexes increase I/O and contention.
- Review which indexes are actually used; identify redundant or overlapping ones.
- Keep only indexes justified by read workload; consider batch-loading approaches and physical design supported by DB.
- Benchmark both read SLA and ingestion throughput before removal/addition.

Follow-up: Would deleting every index fix it? → Maybe writes improve, but crucial lookups, constraints and joins may suffer.

Q4. COMPLEX VIEW STILL TAKES 40 SEC

Question: Finance has a view joining 10 tables and doing daily SUMs. The dashboard is still slow. Why?

Expected interview answer

- A **normal view stores SQL, not results**: the underlying joins and aggregation still run when queried.
- Inspect query plan, join cardinality, filters, indexes/data layout, compute and skew.
- For repeated aggregations, evaluate materialized view / precomputed aggregate table.
- Define refresh interval, incremental update capability, late data handling and reconciliation.
- Enforce proper permissions; a view name alone doesn't secure underlying data.

Follow-up: Can a materialized view be stale? → Yes; freshness depends on refresh semantics and failures.

Q5. JOIN RETURNS EXTRA / MISSING ROWS

Question: Orders have 10M rows and Customers have 2M rows, but an INNER JOIN outputs 12M. Is the relational model broken?

Expected interview answer

- Check actual **relationship and cardinality**: is Customers unique on customer_id? Does it include SCD2 history?
- One-to-many matches can multiply rows; missing customer IDs can drop rows on INNER JOIN.
- Verify primary/business-key uniqueness, foreign-key coverage, nulls, and join predicates.
- For SCD2, join fact.customer_sk to dimension version key (or use event-time valid range).
- Reconcile row counts and totals before/after joining; don't hide issues with DISTINCT.

Follow-up: Are PK/FK constraints always enforced in lakehouses? → Platform-specific; often validate in ETL.

Q6. CLUSTERED KEY vs COMPOSITE INDEX

Question: Orders PK is order_id, but most queries filter on status and date. Would you change the clustered key?

Expected interview answer

- First identify **DBMS** and actual workload; clustered-index behavior differs by engine.
- Do not assume PK always clustered. Measure PK lookups vs status/date range queries and write patterns.
- Consider a non-clustered composite index **(status, order_date)** for selective reports; test filtering/cardinality and leading-column order.
- Clustered key changes affect storage and all secondary index locators in some engines—expensive to change.
- Use plan and benchmark, not universal rules; consider partitioning or columnar reporting system for massive analytics.

Follow-up: Is status low cardinality enough for an index? → Not by itself; selectivity and query conditions matter.

ANSWER FRAMEWORK FOR ALL SIX: Requirement & grain → workload/read/write pattern → concrete design → trade-offs → execution plan + performance measurement → correctness/freshness validation.